

Reading: AWS Lambda

Introduction

Let's build a simple serverless application using AWS Lambda.

This application will have an html front end hosted on AWS Amplify, where you can enter some text. On submitting the form, it will provide you with a response which is capitalized and reverse of your entered text.

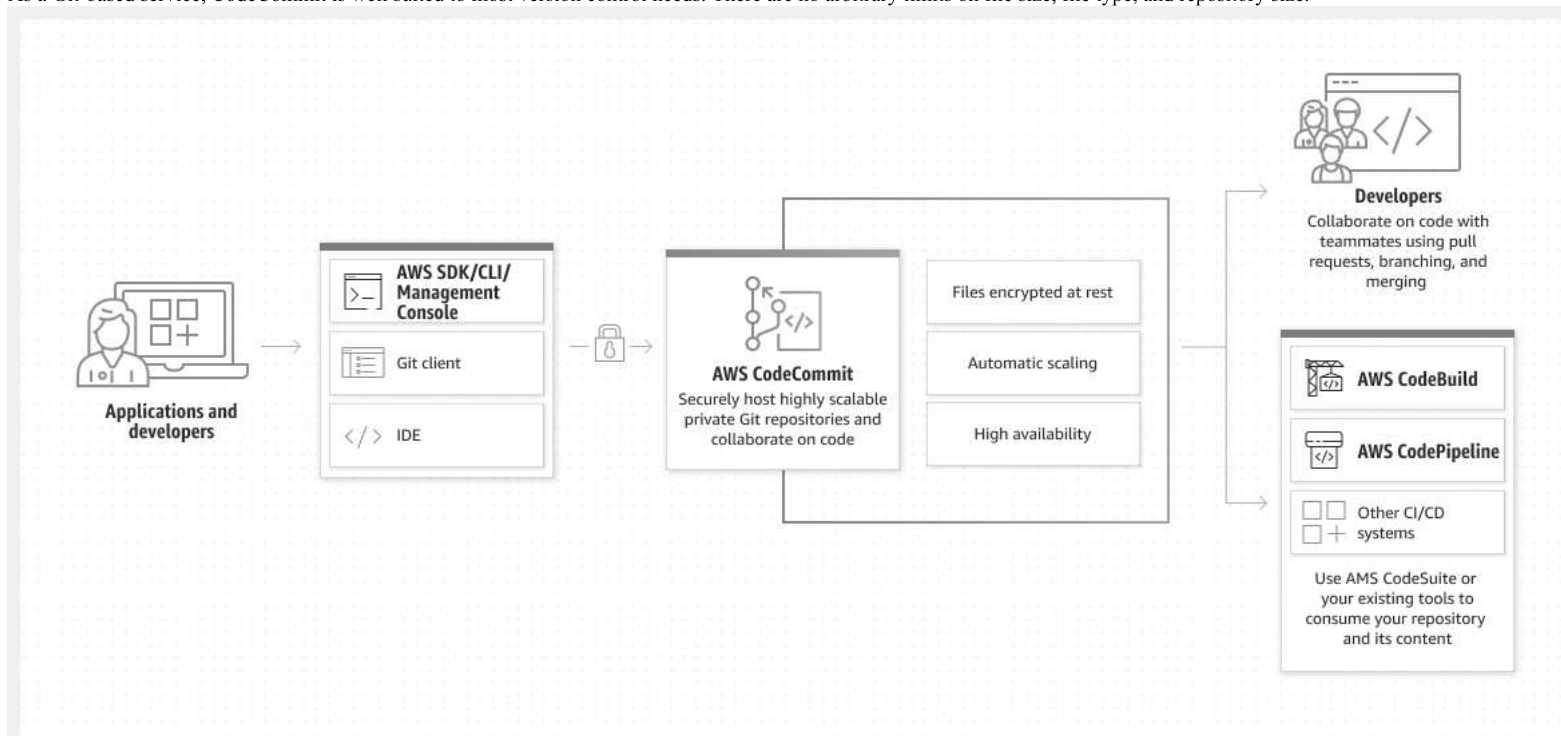
Capitalize and Reverse will be two separate Lambda functions to show you the chaining capabilities. Instead of accessing these functions directly, an API Gateway will be used to accept client requests and respond with the final output.

The application will include two separate Lambda functions: **Capitalize** and **Reverse**. These functions will be chained together using AWS Step Functions. Instead of accessing these functions directly, an API Gateway will be used to accept client requests and respond with the final output.

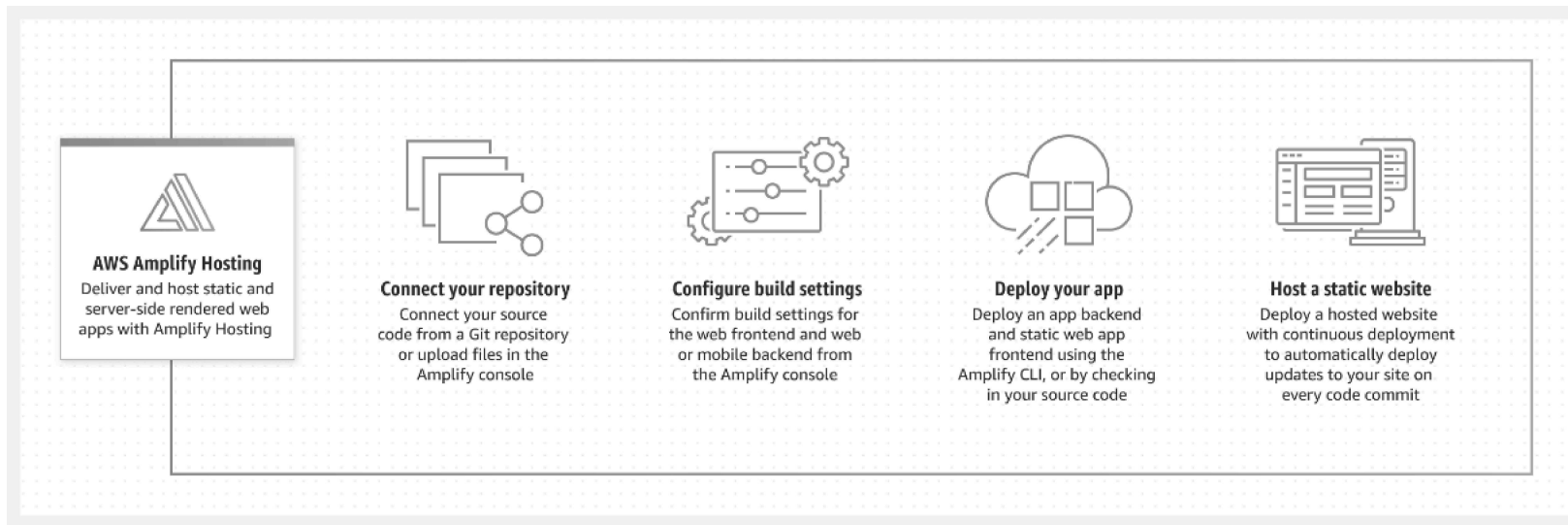
Components Used:

- **AWS CodeCommit:** AWS CodeCommit is a secure, highly scalable, fully managed source control service that hosts private Git repositories.

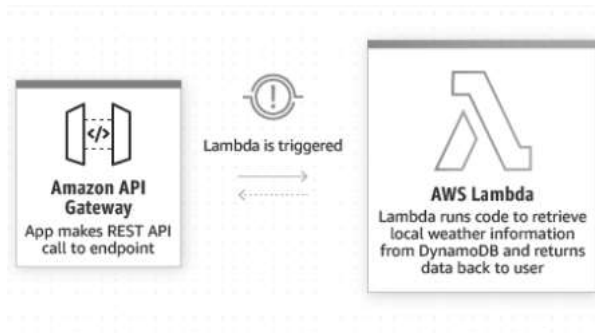
As a Git-based service, CodeCommit is well suited to most version control needs. There are no arbitrary limits on file size, file type, and repository size.



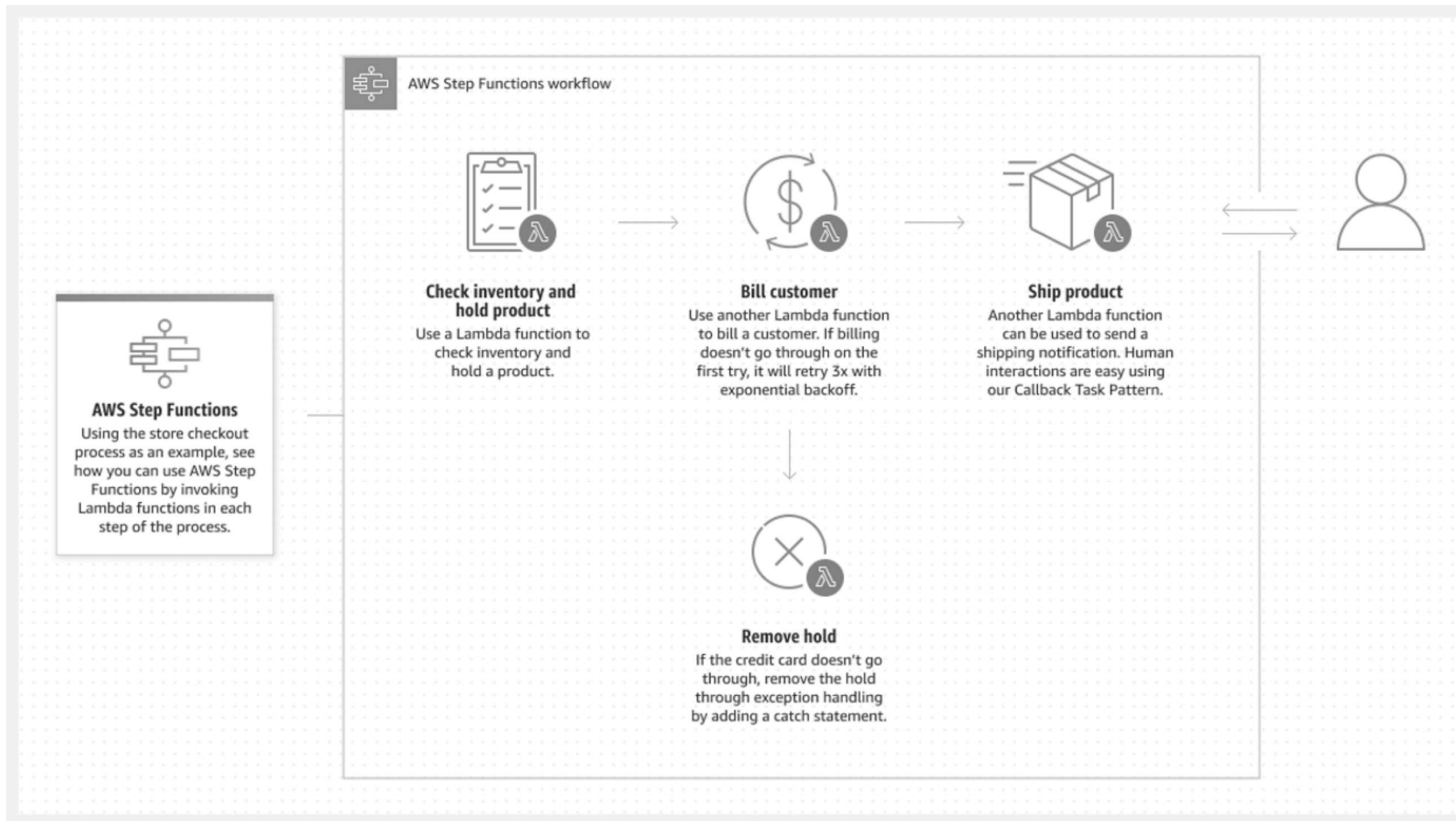
- **AWS Amplify:** AWS Amplify is a complete solution that lets front end web and mobile developers easily build, ship, and host full-stack applications on AWS, with the flexibility to leverage the breadth of AWS services as use cases evolve.



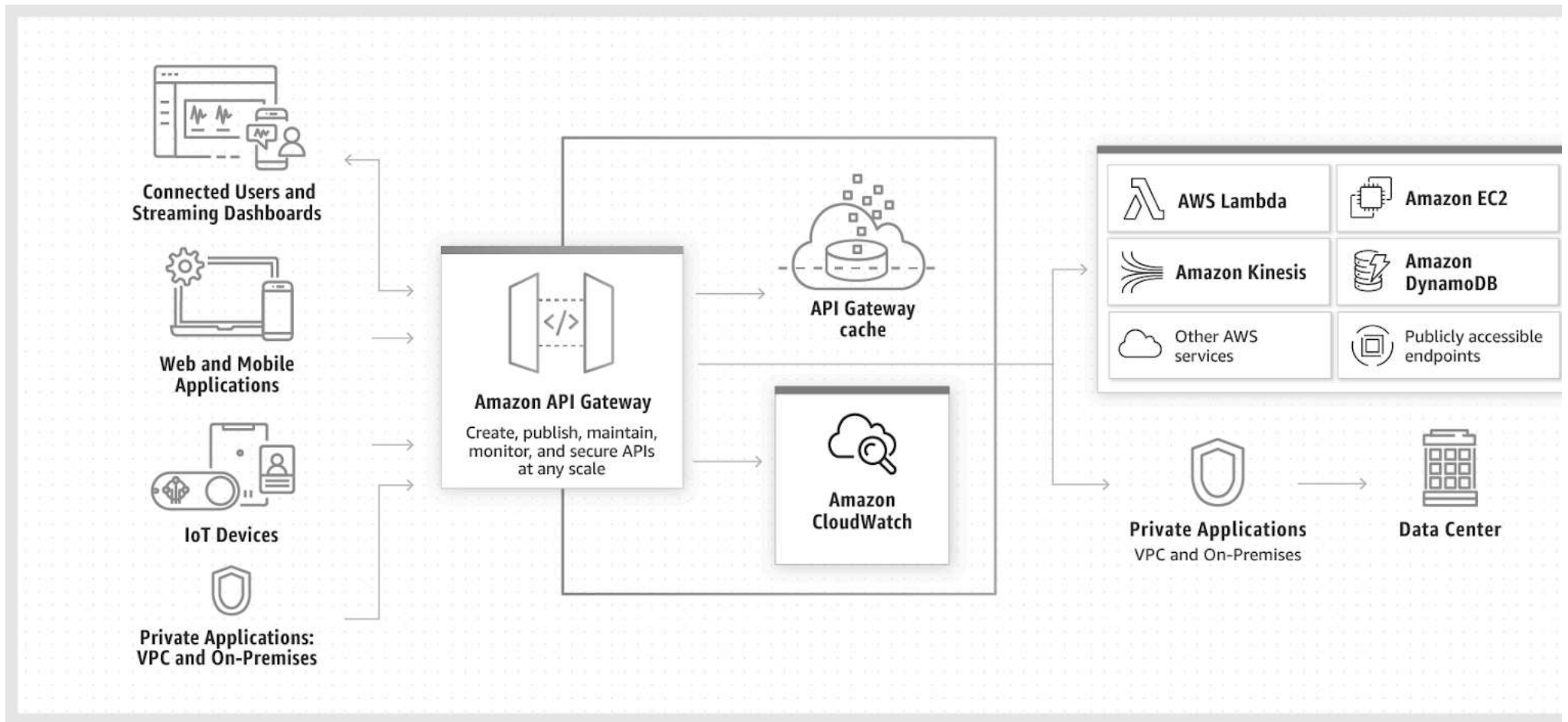
- **AWS Lambda**: AWS Lambda is a serverless, event-driven compute service that lets you run code for virtually any type of application or back end service without provisioning or managing servers. You can trigger Lambda from over 200 AWS services and software as a service (SaaS) application, and only pay for what you use.



- **AWS Step Function**: AWS Step Function is a visual workflow service that helps developers use AWS services to build distributed applications, automate processes, orchestrate microservices, and create data and machine learning (ML) pipelines.

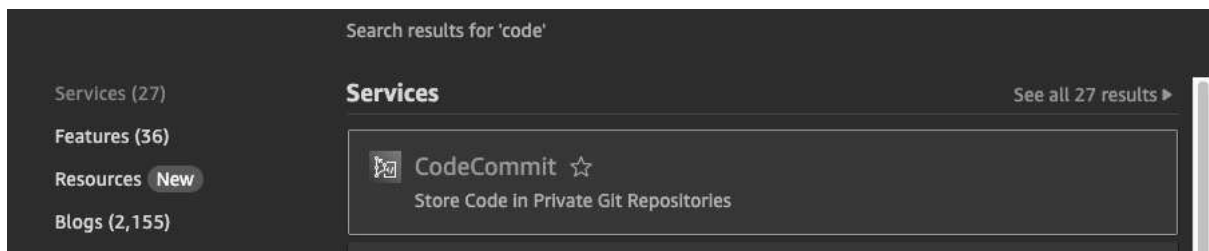


- **AWS API Gateway:** Amazon API Gateway is a fully managed service that makes it easy for developers to create, publish, maintain, monitor, and secure APIs at any scale. APIs act as the "front door" for applications to access data, business logic, or functionality from your back end services.



Process

- Let's start with defining CodeCommit resource which you can use as your code repository.



1. You start with a blank repository. Click on Create repository.

Developer Tools > CodeCommit > Repositories

RepositoriesInfo

Notify

Clone URL

View repository

Delete repository

Create repository

Q

<1>

Name	Description	Last modified
No results		
There are no results to display.		

2. Provide a repository name and an optional description.

about:blank

5/38

Create repository

Create a secure repository to store and share your code. Begin by typing a repository name and a description for your repository. Repository names are included in the URLs for that repository.

Repository settings

Repository name

100 characters maximum. Other limits apply.

Description - *optional*

1,000 characters maximum

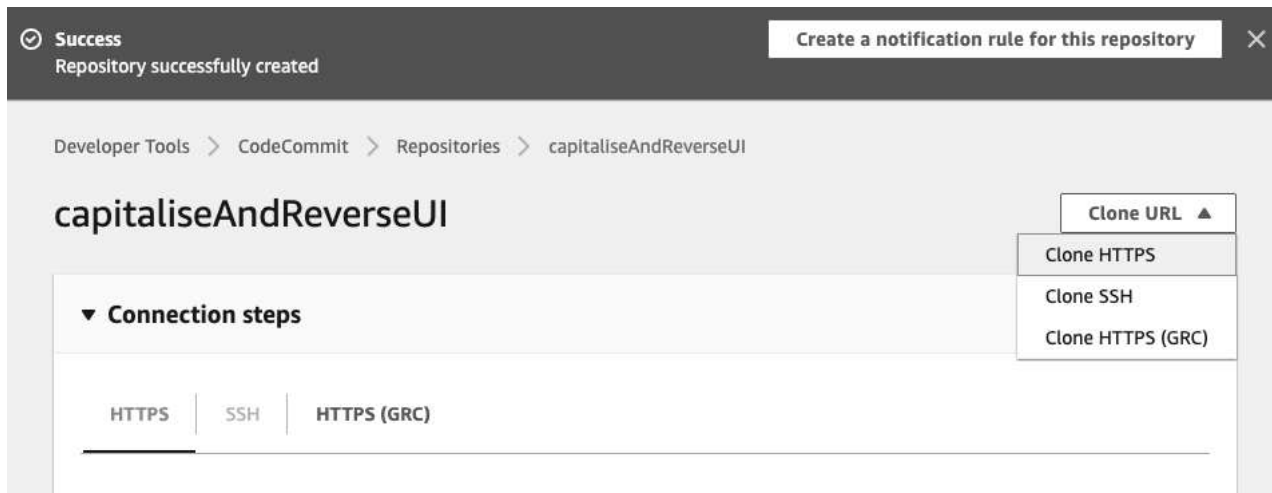
Tags

☐ Enable Amazon CodeGuru Reviewer for Java and Python - *optional*

Get recommendations to improve the quality of the Java and Python code for all pull requests in this repository.

A service-linked role will be created in IAM on your behalf if it does not exist.

3. Now get the details of this repository to clone in your local environment.



4. Clone the repository on your computer to create the required html resources.

```
$ git clone https://git-codecommit.eu-west-2.amazonaws.com/v1/repos/capitaliseAndReverseUI
Cloning into 'capitaliseAndReverseUI'...
Username for 'https://git-codecommit.eu-west-2.amazonaws.com': 
Password for 'https://[redacted]@git-codecommit.eu-west-2.amazonaws.com': 
warning: You appear to have cloned an empty repository.
```

5. You then create a simple html page (that will contain the require JavaScript and CSS sections).

```
$ git add index.html
$ git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   index.html
```

6. Commit your changes and you can also push the changes to the remote repository on AWS CodeCommit.

```
$ git commit -m "index.html created which accepts a text value"
[master (root-commit) 14aa3ab] index.html created which accepts a text value
1 file changed, 49 insertions(+)
 create mode 100644 index.html
```

Host Front-End with AWS Amplify

1. Now create AWS Amplify resource to host your static content (HTML).

Search results for 'aws Ampli'

Services (85)

Features (200)

Resources New

Blogs (18,041)

Knowledge Articles (30)

Tutorials (101)

Events (466)

Marketplace (7,564)

Services

See all 85 results ▶

 AWS Amplify ☆
AWS Amplify is a complete platform—frameworks & tools and app services—for dev...

Top features

All apps

 AWS Signer ☆
Ensuring trust and integrity of your code

2. Start by creating the resource.



AWS Amplify

Fastest, easiest way to develop mobile and web apps that scale.

GET STARTED



AWS Amplify is a set of products and tools that enable mobile and front-end web developers to build and deploy secure, scalable full-stack applications, powered by AWS.

3. Choose Host your web app.

Amplify Hosting



Host your web app

Connect your Git repository to continuously deploy your frontend and backend. Host it on a globally available CDN.



Get started

4. Select AWS CodeCommit; this is where you have pushed changes from your local environment to the repository.

Get started with Amplify Hosting

Amplify Hosting is a fully managed hosting service for web apps. Connect your repository to build, deploy, and host your web app.

From your existing code

Connect your source code from a Git repository or upload files to host a web app in minutes.

☐ GitHub



☐ Bitbucket



☐ GitLab



☒ AWS CodeCommit



☐ Deploy without Git provider



Amplify Hosting requires read-only access to your repository.

Continue

5. You will now link the master branch with AWS Amplify. This will provide the continuous delivery for you whenever you push changes to master branch.

Add repository branch

AWS CodeCommit

 AWS CodeCommit authorization was successful.

Repository service provider

 AWS CodeCommit

Recently updated repositories

If you don't see your repository below, please push a commit and then click the refresh button.

capitaliseAndReverseUI ▼ 

Branch

Select a branch from your repository.

master ▼

☐ Connecting a monorepo? Pick a folder.

Cancel Previous Next

6. Accept the default build settings.

Build settings

App build and test settings

App name

Pick a name for your app.

capitaliseAndReverseUI

Name cannot contain periods

Build and test settings

We've auto-detected your app's build settings. Please ensure your build command and output folder (baseDirectory) are correctly detected.

1 version: 1

2 frontend:

3 phases:

4 # IMPORTANT - Please verify your build commands

5 build:

6 commands: ☐

7 artifacts:

8 # IMPORTANT - Please verify your build output directory

9 baseDirectory: /

10 files:

11 - '**/*'

12 cache:

13 paths: ☐

14

Build and test settings

Download

Edit

☒ Allow AWS Amplify to automatically deploy all files hosted in your project root directory

► Advanced settings

IAM Role

IAM service role

Amplify requires read-only access to your CodeCommit repository. To create custom roles go to the IAM console [↗](#).

☒ Create and use a new service role

☐ Use an existing service role

Cancel

Previous

Next

7. Review and complete the process.

Review

Repository details

Repository service AWS CodeCommit	Branch environment
Repository capitaliseAndReverseUI	Application root
Branch master	

App settings

Edit

App name capitaliseAndReverseUI	Framework Web
Build image Using default image	Build settings Auto-detected settings will be used
Environment variables None	

Cancel

Previous

Save and deploy

8. Process takes some time to complete (provisioning, building, and deploying your changes).

capitaliseAndReverseUI

Actions ▾

The app homepage lists all deployed frontend and backend environments.

► Learn how to get the most out of Amplify Hosting

0 of 5 steps complete X

Hosting environments

Backend environments

This tab lists all connected branches, select a branch to view build details.

Connect branch

master

Continuous deploys set up (Edit)



https://master...amplifyapp.com

Provision

Build

Deploy

Last deployment

23/11/2022, 22:05:05

Last commit

This is an autogenerated message | Auto-build | AWS CodeCommit - master

Previews

Disabled

9. Once completed, you can visit the URL to see your web application in action.

← → ↺

master.amplifyapp.com

🔒

📄 ☆

⋮

Enter some text here

Submit

Deploying Your Application on AWS Amplify using GitHub

- 1. Select GitHub and click Next. You'll need to define the GitHub repository you want to use as your code repository.

The screenshot shows the AWS Amplify console interface. At the top, there's a navigation bar with the AWS logo, 'Services', a search bar, and a user profile 'saphashree' in 'N. Virginia'. Below this, the 'AWS Amplify' header includes 'All apps' and a 'Create new app' button. On the right, there are links for 'Support' and 'Docs'. A left-hand sidebar contains a progress indicator with four steps: 1. Choose source code provider (selected), 2. Add repository and branch, 3. App settings, and 4. Review.

The main content area is titled 'Start building with Amplify' and includes a sub-header 'Deploy your app'. It provides instructions on how to deploy an app from a Git provider. Four options are presented in a row: GitHub, BitBucket, CodeCommit, and GitLab. The GitHub option is highlighted with a grey background. Below these options, a note states 'Amplify requires read-only access to your repository.' and a link to 'Deploy without Git' is provided. At the bottom right of the main area are 'Cancel', 'Previous', and 'Next' buttons. A footer bar contains a question mark icon and the text 'Looking to build an app with our Gen 1 tools (Amplify Studio/Amplify CLI)?' followed by a link 'Create an app with Gen 1'.

Note: After selecting GitHub, You'll be prompted to authorize access.

2. Choose the project you wish to deploy, then connect its working branch (main/master) to GitHub. This setup will enable continuous delivery, automatically deploying updates whenever you push changes to the working branch.

 Services [Alt+S] 

All apps / Create new app

✓ Choose source code provider

● Add repository and branch

3 App settings

4 Review

Add repository and branch

If you don't see your repository in the dropdown above, ensure the Amplify GitHub App has permissions to the repository. If your repository still doesn't appear, push a commit and click the refresh button.



☐ My app is a monorepo

Note: Specify the Frontend build command and the build directory according to your project.

 Services [Alt+S]

All apps / Create new app

✓ Choose source code provider

✓ Add repository and branch

App settings

4 Review

App settings

App name

Build settings

Your build settings have been detected automatically, please verify your "Frontend build command" and "Build output directory".

Auto-detected frameworks

Frontend build command	Build output directory
	/

Edit YML file

☐ Password protect my site

Advanced settings

CloudShell Feedback © 2024, Amazon Web Services

3. Review and complete the process.

aws

Services

Search

[Alt+S]

All apps / Create new app

✓

Choose source code provider

✓

Add repository and branch

✓

App settings

Review

github

Branch

master

Manvi1113/vftvk-Simple-Interest-Calculator

Monorepo app root

App settings

App name

vftvk-Simple-Interest-Calculator

Framework

None

Frontend build command

Build output directory

/

Advanced settings

Build image

Using default image

Live package updates

Keep cookies in cache key

Enabled

Environment variables

None

Server-Side Rendering (SSR) deployment

Disabled

First-time account setup required

Amplify needs to run a one-time setup for this account and region before it can deploy resources in the account. This process will begin when you click "Save and deploy" and usually takes be

CloudShell

Feedback

© 2024, Amazon Web Ser

The process takes some time to complete (provisioning, building, and deploying your changes).

aws

Services

Search

[Alt+S]

All apps / vftvk-Simple-Interest-Calculator / Overview

vftvk-Simple-Interest-Ca... <<

Overview

Hosting

App settings

vftvk-Simple-Interest-Calculator

App ID: d39fzp1u1jfw1

Production branch

master >

Deployed

Domain

https://master.d39fzp1u1jfw1.amplifyapp.com

Updated

8/21/2024, 3:17 PM

Last commit

Auto-build

Repository

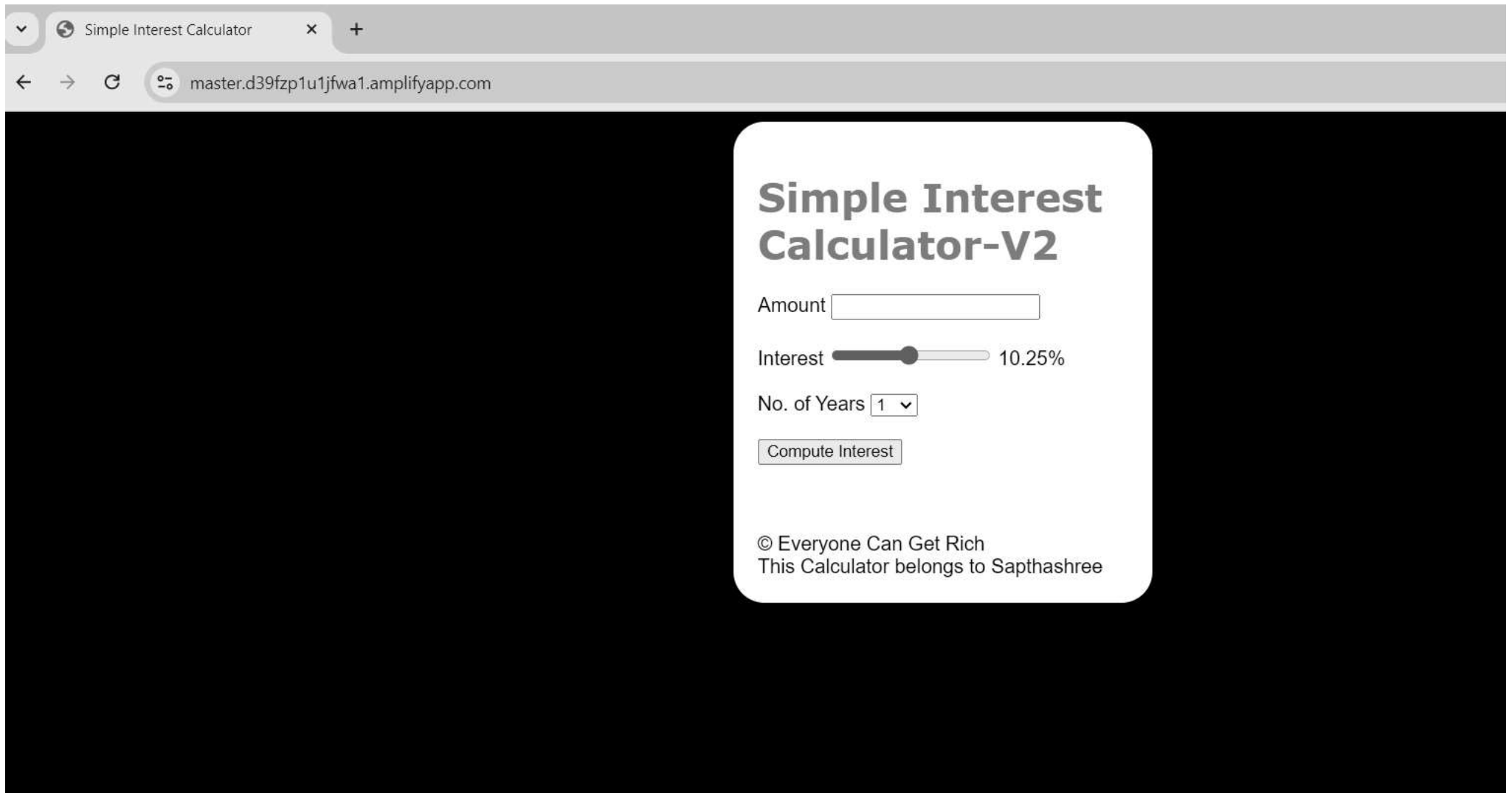
vftvk-Simple-Interest-Calculator:master

Other branches 0

Search...

No other branches added. [Add branch](#)

4. Once completed, you can visit the provided URL to see your web application in action.



Note: If the deployment fails, click on the Hosting from the left pannel and choose build settings and update the YML file.

But this application is not complete, you are yet to build the back end to do the capitalization and reversal of the input string.

Create AWS Lambda Functions

Capitalize Function:

1. You start by defining the first AWS Lambda function to Capitalize the input text.

ServicesSee all 6 results ►

 **Lambda** ☆
Run Code without Thinking about Servers

Compute

AWS Lambda

lets you run code without thinking about servers.

You pay only for the compute time that you consume — there is no charge when your code is not running. With Lambda, you can run code for virtually any type of application or backend service, all with zero administration.

Get started

Author a Lambda function from scratch, or choose from one of many preconfigured examples.

[Create a function](#)

2. Provide the function name and runtime. Choose Python 3.9 for this.

Create function

Info

AWS Serverless Application Repository applications have moved to Create application.

Author from scratch

Start with a simple Hello World example.

Use a blueprint

Build a Lambda application from sample code and configuration presets for common use cases.

Container image

Select a container image to deploy for your function.

Basic information

Function name

Enter a name that describes the purpose of your function.

capitaliseFunc

Use only letters, numbers, hyphens, or underscores with no spaces.

Runtime

Info

Choose the language to use to write your function. Note that the console code editor supports only Node.js, Python, and Ruby.

Python 3.9

Architecture

Info

Choose the instruction set architecture you want for your function code.

x86_64

arm64

Permissions

Info

By default, Lambda will create an execution role with permissions to upload logs to Amazon CloudWatch Logs. You can customize this default role later when adding triggers.

Change default execution role

Advanced settings

Cancel

Create function

3. Defining the function will look like this:

about:blank

21/38

The screenshot displays the AWS Lambda console interface for a function named 'capitaliseFunc'. The top section, 'Function overview', includes a card for the function itself and a 'Layers' section with '(0)' layers. Below this is an 'API Gateway' card and a '+ Add trigger' button. The bottom section, 'Code source', shows a code editor with a file named 'lambda_function.py'. The code in the editor is as follows:

```
1 import json
2
3 def lambda_handler(event, context):
4     input_text = str(event['inputText'])
5     capitalised_input_text = input_text.upper()
6     return {"inputText": capitalised_input_text}
```

4. The code you have written is very basic, as it accepts input text as part of the body (it's a HTTP POST function). And returns the object again as input text with capitalized value (so you can chain this to the reverse function).

```
import json
def lambda_handler(event, context):
    input_text = str(event['inputText'])
    capitalised_input_text = input_text.upper()
    return {"inputText": capitalised_input_text}
```

Configure test event

A test event is a JSON object that mocks the structure of requests emitted by AWS services to invoke a Lambda function. Use it to see the function's invocation result.

To invoke your function without saving an event, modify the event, then choose Test. Lambda uses the modified event to invoke your function, but does not overwrite the original event until you choose Save changes.

Test event action

Create new event

Edit saved event

Event name

capitaliseText



Delete

Event JSON

Format JSON

```
1 {  
2   "inputText": "value1"  
3 }
```

5. And once you deploy your function, you can then test it and see the following outcome.

ToolsWindow

TestDeploy

lambda_functionExecution result

Execution results

Status: SucceededMax memory used: 36 MBTime: 1.29 ms

Test Event Name

capitaliseText

Response

```
{  
  "inputText": "VALUE1"  
}
```

Function Logs

START RequestId: b56e0c53-6fd2-4080-9800-ac42efdb43b4 Version: \$LATEST
END RequestId: b56e0c53-6fd2-4080-9800-ac42efdb43b4
REPORT RequestId: b56e0c53-6fd2-4080-9800-ac42efdb43b4 Duration: 1.29 ms Billed Duration: 2 ms Memory Size: 128 MB Max Memory Used: 36 MB Init Duration: 106.76 ms

Request ID

b56e0c53-6fd2-4080-9800-ac42efdb43b4

6. Similarly, you create the reverse function.

lambda_function × Execution results × +

```
1 import json
2
3 def lambda_handler(event, context):
4     input_text = str(event['inputText'])
5     reversed_input_text = input_text[::-1]
6     return {"inputText": reversed_input_text}
7
```

```
import json
def lambda_handler(event, context):
    input_text = str(event['inputText'])
    reversed_input_text = input_text[::-1]
    return {"inputText": reversed_input_text}
```

7. Deploy and test the function.

lambda_function × Execution result: × +

▼ Execution results

Test Event Name
reverseText

Response

```
{
  "inputText": "1eulav"
}
```

8. Now that you have two functions defined and created, you can chain them together using StepFunctions.

 **Step Functions** ☆
Coordinate Distributed Applications

Top features
[State Machines](#) [Activities](#) [Getting started](#) [Sample projects](#) [Feature spotlight](#)

9. Start by creating a state machine.

State machines (1)  [View details](#) [Edit](#) [Copy to new](#) [Delete](#) [Create state machine](#)

Execution counts are based on the most recent 1000 executions

Any type ▼ < 1 > 

Name ▼	Type ▼	Creation date ▼	Status	Logs	Total	Running	Succeeded	Failed	Timed out	Aborted
--------	--------	-----------------	--------	------	-------	---------	-----------	--------	-----------	---------

10. You can choose to design workflow visually for ease and use Express to make your functions work synchronously.

Choose authoring method

Design your workflow visually

Drag and drop your workflow together with Step Functions Workflow Studio. **New**

Write your workflow in code

Author your workflow using Amazon States Language. You can generate code snippets to easily build out your workflow steps.

Run a sample project

Deploy and run a fully functioning sample project in minutes using CloudFormation.

Type

☐ **Standard**

Durable, checkpointed workflows for machine learning, order fulfillment, IT/DevOps automation, ETL jobs, and other long-duration workloads.

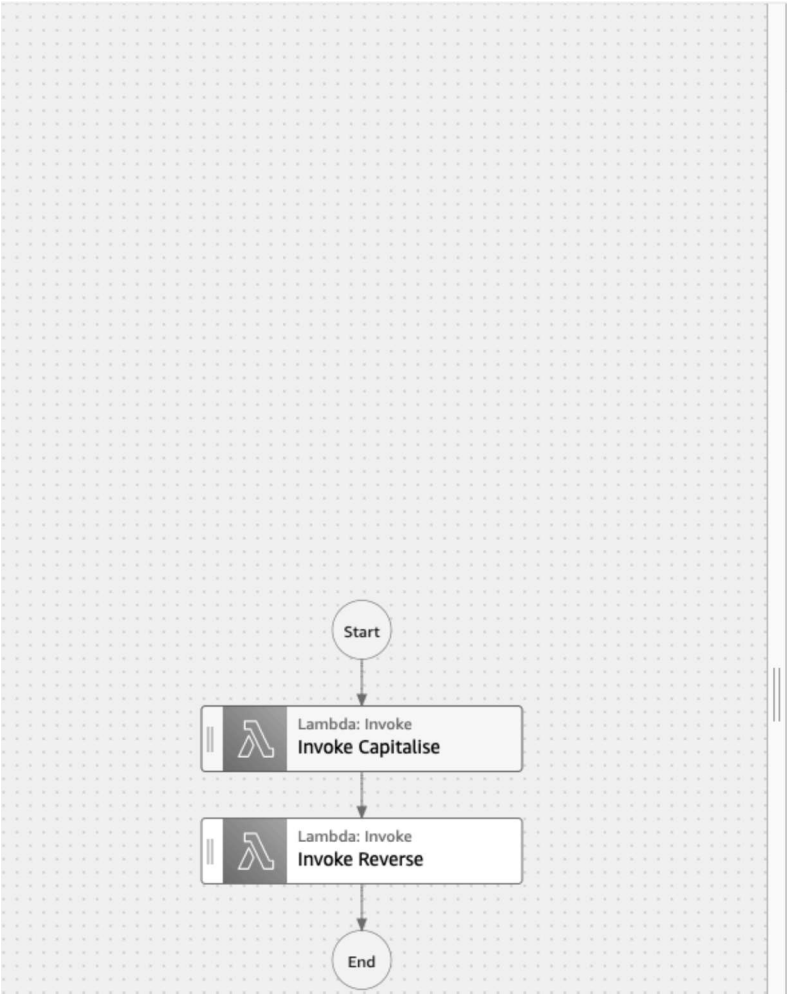
☒ **Express**

Event-driven workflows for streaming data processing, microservices orchestration, IoT data ingestion, mobile backends, and other short duration, high-event-rate workloads.

► **Help me decide**

Cancel

Next



Invoke Capitalise

Configuration

Input

Output

Error handling

State name

Invoke Capitalise

API

Lambda: Invoke

Integration type

Info

The type of service integration to use. [Learn more](#)

Optimized

API Parameters

Edit as JSON

Function name

The Lambda function to invoke

Enter function name

arn:aws:lambda:eu-west-2::function:capitaliseFunc:\$LATEST

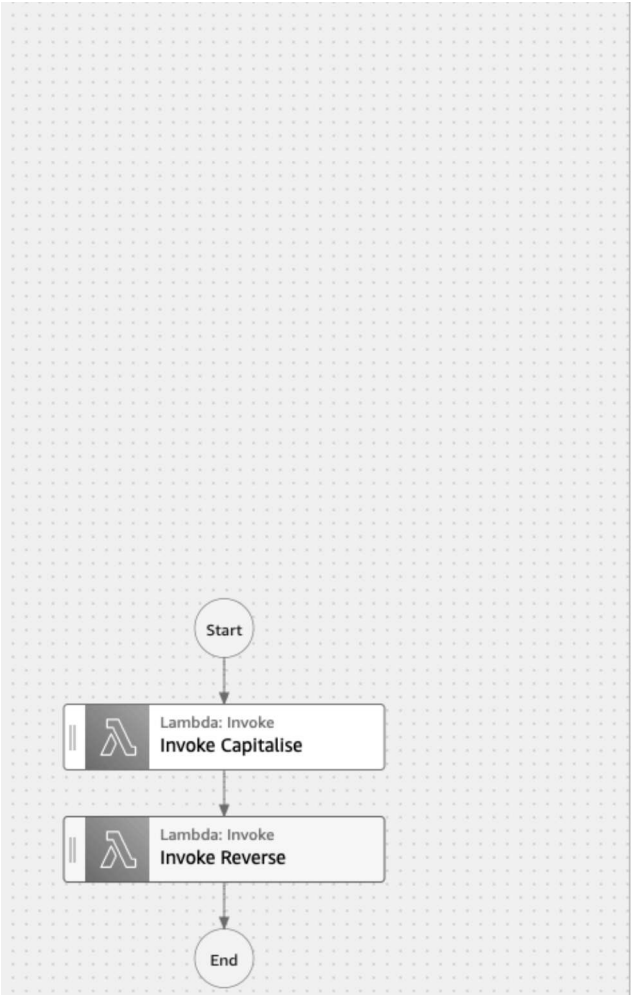
Must be a valid function name.

View function

Payload

The JSON that you want to provide to your Lambda function.

Use state input as payload



Invoke Reverse

Configuration

Input

Output

Error handling

State name

Invoke Reverse

API

Lambda: Invoke

Integration type Info

The type of service integration to use. [Learn more](#)

Optimized

API Parameters

Edit as JSON

Function name

The Lambda function to invoke

Enter function name

arn:aws:lambda:eu-west-2: :function:reverseFunc:\$LATEST

Must be a valid function name.

View function

Payload

The JSON that you want to provide to your Lambda function.

Use state input as payload

about:blank

27/38

Cancel

Start execution

Save

Definition

Define your workflow using Amazon States Language . Test your data flow with the new Data Flow Simulator.

Export ▼

Layout ▼

Generate code snippet ▼

Format JSON

1 ▼

{

2

"Comment": "A description of my state machine",

3

"StartAt": "Invoke Capitalise",

4 ▼

"States": {

5 ▼

"Invoke Capitalise": {

6

"Type": "Task",

7

"Resource": "arn:aws:states:::lambda:invoke",

8

"OutputPath": "\$.Payload",

9 ▼

"Parameters": {

10

"Payload.\$": "\$",

11

"FunctionName": "arn:aws:lambda:eu-west-2: :function:capitaliseFunc:\$LATEST"

12

},

13 ▼

"Retry": [

14 ▼

{

15 ▼

"ErrorEquals": [

16

"Lambda.ServiceException",

17

"Lambda.AWSLambdaException",

18

"Lambda.SdkClientException",

19

"Lambda.TooManyRequestsException"

20

],

21

"IntervalSeconds": 2,

22

"MaxAttempts": 6,

23

"BackoffRate": 2

24

}

25

],

26

"Next": "Invoke Reverse"

27

},

28 ▼

"Invoke Reverse": {

29

"Type": "Task",

30

"Resource": "arn:aws:states:::lambda:invoke",

31

"OutputPath": "\$.Payload",

32 ▼

"Parameters": {

33

"Payload.\$": "\$",

34

"FunctionName": "arn:aws:lambda:eu-west-2: :function:reverseFunc:\$LATEST"

35

},

36 ▼

"Retry": [

37 ▼

{

↺

+

−

⦿

Workflow Studio **New**

Start

Invoke Capitalise

Invoke Reverse

End

11. Click on New execution to test your State machine..

Edit state machine

Export

New execution

Start execution

Start an execution using the latest definition of the state machine. [Learn more](#)

Name - optional

testint_state_machine

Input - optional

Enter input values for this execution in JSON format

Format JSON

Export

Import

1

```
{"inputText": "this is an example of anagram radar"}
```

Execution: testing_state_machine:d0f5e86c-20c6-45b9-985b-eadd2dc30b03

Edit state machine

Export

New execution

Details

Execution input and output

Definition

Input

1

{

2

"inputText": "this is an example of anagram radar"

3

}

Output

1

{

2

"inputText": "RADAR MARGANA FO ELPMAXE NA SI SIHT"

3

}

Create API Gateway

Search results for 'API Gate'

Services

See all 37 results

API Gateway

Build, Deploy and Manage APIs

REST API

Develop a REST API where you gain complete control over the request and response along with API management capabilities.

Works with the following:

Lambda, HTTP, AWS Services

Import

Build

about:blank

29/38

Choose the protocol

Select whether you would like to create a REST API or a WebSocket API.

☒ REST ☐ WebSocket

Create new API

In Amazon API Gateway, a REST API refers to a collection of resources and methods that can be invoked through HTTPS endpoints.

☒ New API ☐ Import from Swagger or Open API 3 ☐ Example API

Settings

Choose a friendly name and description for your API.

API name*

capitaliseAndReverseAPI

Description

Endpoint Type

Edge optimized

* Required

Create API

APIs

Custom Domain Names

VPC Links

API: capitaliseAndR...

Resources

Stages

Authorizers

Resources

/

Actions

RESOURCE ACTIONS

Create Method

Create Resource

Enable CORS

Edit Resource Documentation

API ACTIONS

Deploy API

Import API

Edit API Documentation

Delete API

/ Methods

Resources

Actions

New Child Resource

/

Use this page to create a new child resource for your resource.

Configure as

☒ proxy resource

☐

Resource Name*

capitaliseAndReverse

Resource Path*

/ capitaliseandreverse

Enable API Gateway CORS

☐

You can add path parameters using brackets. For example, the resource path {username} represents a path parameter called 'username'. Configuring /{proxy+} as a proxy resource catches all requests to its sub-resources. For example, it works for a GET request to /foo. To handle requests to /, add a new ANY method on the / resource.

* Required

Cancel

Create Resource

Resources

Actions

/capitalise Methods

RESOURCE ACTIONS

Create Method

Create Resource

Enable CORS

Edit Resource Documentation

Delete Resource

Options

None

Not required

Provide information about the target backend that this method will call and whether the incoming request data should be modified.

Integration type

☐ Lambda Function ⓘ

☐ HTTP ⓘ

☐ Mock ⓘ

☒ AWS Service ⓘ

☐ VPC Link ⓘ

AWS Region

eu-west-2 ⓘ

AWS Service

Step Functions ⓘ

AWS Subdomain

ⓘ

HTTP method

POST ⓘ

Action

StartSyncExecution ⓘ

Execution role

arn:aws:iam:: /APIGatewayToStepFunctions ⓘ

Credentials cache

Do not add caller credentials to cache key ⓘ

Content Handling

Passthrough ⓘ ⓘ

Use Default Timeout

☒ ⓘ

▼ Mapping Templates ●

- Request body passthrough**
- ☐ When no template matches the request Content-Type header ⓘ
 - ☐ When there are no templates defined (recommended) ⓘ
 - ☒ Never ⓘ

Content-Type	
application/json	⊖

⊕ Add mapping template

application/json

Generate template:

```
1 #set($input = $input.json('$'))
2 {
3   "input": "$util.escapeJavaScript($input)",
4   "stateMachineArn": "arn:aws:states:eu-west-2::stateMachine
   :CapitaliseAndReverseStateMachine"
5 }
```

You then define the Stage. A Stage is a named reference to a deployment, which is a snapshot of the API. You use a Stage to manage and optimize a particular deployment. For example, you can configure Stage settings to enable caching, customize request throttling, configure logging, define stage variables, or attach a canary release for testing.

prod Stage Editor

Delete Stage

Configure Tags

Invoke URL: https://[redacted]execute-api.eu-west-2.amazonaws.com/prod

- Settings
- Logs/Tracing
- Stage Variables
- SDK Generation
- Export
- Deployment History
- Documentation History
- Canary

Cache Settings

Enable API cache ☐

Default Method Throttling

Choose the default throttling level for the methods in this stage. Each method in this stage will respect these rate and burst settings. Your current account level throttling rate is **10000** requests per second with a burst of **5000** requests. Read more about API Gateway throttling

Enable throttling ☒ ⓘ

Rate requests per second

Burst requests

Web Application Firewall (WAF) [Learn more.](#)

Select the Web ACL to be applied to this stage.

Web ACL

None ▼

[Create Web ACL](#)

Client Certificate

Select the client certificate that API Gateway will use to call your integration endpoints in this stage.

Certificate

None ▼

Save Changes

Generate the SDK, so you can use the generated code in your web app and call this API Gateway.

Settings

Logs/Tracing

Stage Variables

SDK Generation

Export

Deployment History

Documentation History

Canary

Choose a platform and provide the settings for the SDK you will generate.

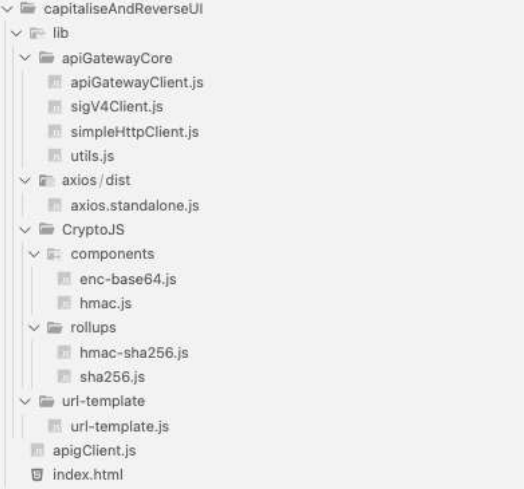
Platform*

JavaScript

* Required

Generate SDK

You then extract the generated JavaScript code as below:



And finally deploy the API (back in the AWS API Gateway section).

Deploy API

Choose a stage where your API will be deployed. For example, a test version of your API could be deployed to a stage named beta.

Deployment stage

[New Stage]

Stage name*

prod

Stage description

Deployment description

Cancel

Deploy

Finalize Front-End

Your final HTML will look like below; do notice that you have introduced a field to display your output

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta http-equiv="X-UA-Compatible" content="IE=edge" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Reverse and Capitalise with AWS Lambda</title>
    <style>
      body { font-family: Verdana; text-align: center; }
      form { max-width: 500px; margin: 50px auto; padding: 30px 20px; box-shadow: 2px 5px 10px rgba(0, 0, 0, 0.5); }
      .form-control { text-align: left; margin-bottom: 25px; }
      .form-control input { padding: 10px; display: block; width: 95%; }
    </style>
  </head>
  <body>
    <form id="form" onsubmit="callLambdaFunction(); return false;">
      <div class="form-control">
        <input type="text" id="inputText" placeholder="Enter some text here" />
      </div>
      <div class="form-control">
        <button type="submit" value="submit">Submit</button>
      </div>
      <div class="form-control">
        <input type="text" readonly id="outputText" placeholder="Output will appear here" />
      </div>
    </form>
    <script type="text/javascript" src="lib/axios/dist/axios.standalone.js"></script>
    <script type="text/javascript" src="lib/CryptoJS/rollups/hmac-sha256.js"></script>
    <script type="text/javascript" src="lib/CryptoJS/rollups/sha256.js"></script>
    <script type="text/javascript" src="lib/CryptoJS/components/hmac.js"></script>
    <script type="text/javascript" src="lib/CryptoJS/components/enc-base64.js"></script>
    <script type="text/javascript" src="lib/url-template/url-template.js"></script>
    <script type="text/javascript" src="lib/apiGatewayCore/sigv4Client.js"></script>
    <script type="text/javascript" src="lib/apiGatewayCore/apiGatewayClient.js"></script>
    <script type="text/javascript" src="lib/apiGatewayCore/simpleHttpClient.js"></script>
    <script type="text/javascript" src="lib/apiGatewayCore/utills.js"></script>
    <script type="text/javascript" src="apigClient.js"></script>
    <script type="text/javascript">
      function callLambdaFunction() {
        try {
          var inputTextValue = document.getElementById("inputText").value;
          var apigClient = apigClientFactory.newClient();
          var params = {};
          var body = { inputText: inputTextValue };
          apigClient.capitaliseandreversePost(params, body)
            .then(function (result) {
              document.getElementById("outputText").value = JSON.parse(result.data.output).inputText;
            })
            .catch(function (result) {
              console.log(result);
            });
        } catch (error) {
          console.log(error);
        }
        return false;
      }
    </script>
  </body>
</html>
```

You then commit and push the changes to AWS CodeCommit repository and wait for it to be deployed.

capitaliseAndReverseUI

Actions

The app homepage lists all deployed frontend and backend environments.

Learn how to get the most out of Amplify Hosting

1 of 5 steps complete

Hosting environments

Backend environments

This tab lists all connected branches, select a branch to view build details.

Connect branch

master

Continuous deploys set up (Edit)



https://master...amplifyapp.com

Provision

Build

Deploy

Last deployment

24/11/2022, 00:39:02

Last commit

Please visit AWS CodeCommit Co... | 74aedc0 | AWS CodeCommit - master

Previews

Disabled

And you can now test your web app by visiting the URL provided to you by AWS Amplify.

about:blank

37/38

Conclusion

Services provided by AWS, more specifically around Lambda can be used to create sophisticated applications providing both front and back end. And you can build the whole eco system for your app from code repository to deployed as a serverless application.



Skills Network